Problem Statement

Most LLMs / QA systems trained on mythological or historical texts fail to provide precise, verifiable answers due to:

- Paraphrased or modernized sources (Ex: biased blogs)
- Poor context resolution across non-adjacent paragraphs
- Inability to preserve relationships and events authentically

RamayanaGPT addresses this by delivering **accurate, unbiased responses** directly rooted in *Valmiki's Ramayana*, using a **compact 25MB Knowledge Graph** (KG) and efficient RAG-based architecture. The system aims to preserve the sanctity and structure of original scripture while being lightweight and interpretable.

Approaches and Pipelines

Initial Approach: MiniRAG + KG + MainRAG (architecture)

- **Data Preparation**: Original Valmiki Ramayan is processed via an LLM to:

    1. Normalize narration (perspective differences and slight toxicity rephrasing)
    2. Extract events and entities
    3. Generate **Knowledge Graph Triplets** + Modified Ramayana

- **Query Flow**:

    1. **MiniRAG** retrieves rough chunks from Modified Ramayana
    2. **MainRAG** refines chunks using context and query
    3. **Knowledge Graph** aids entity resolution and feeds triplets
    4. Combined info is sent to the **LLM** for final generation

Didn't work well, because it:

- Operated like a search bot retrieving similar paragraphs
- Failed to correlate events across scattered parts (Ex: can't identify any semantic similarity between Rama in ayodhdhya kanda and yuddha kanda)
- Couldn't fully utilize the graph's semantic depth (only used graph for Relations between characters)

Final Approach: KG-first, RAG-second (architecture)

- Each **Kanda** has its own **Knowledge Graph** (~7 total), avoiding memory overload
- The KG handles **primary retrieval (1-hop triplets)** based on query entity or relation
- RAG acts as a **contextual enhancer** if required, not the core retriever
- **Triplets include metadata** mapping them to original chunks (backtrackable)
- This avoids duplication and semantic overlap, enabling better linking of related events

Improvements:

- Enabled accurate, relationship-aware responses
- Events, traits, and dependencies are preserved in graph structure

---

Aspect	**What Worked**	**What Didn't**
KG Usage	Representing the story as nodes and edges allowed for better tracking of relationships, traits, and resolutions	Graphs are large and not memory-efficient if not split
MiniRAG + MainRAG	Layered chunking seemed promising	Couldn't correlate multi-paragraph logic/events
Triplet Metadata	Helped map triplets back to source	Multiple source collisions due to duplication
Final Graph-RAG Pipeline	Clean 1-degree traversal with optional context	Adds RAG complexity with only minor gains sometimes

Final Working Query Pipeline: link

Query → Optional RAG Refinement → KG (1-hop traversal) → Get Triplets + Linked Chunks → LLM → Response

- **KG per Kanda**: Prevents overload and duplication
- **Triplet metadata**: Tracks provenance
- **Optional RAG**: Adds context only if necessary

CONCLUSIONS

Retrospective Changes That Didn't Help Much

Change	Outcome
Multi-hop traversal	Added noise; 1-hop already retrieves most needed info
Additional context via Main RAG	Often redundant if KG triplets are clean
Full text in graph metadata	High memory + collision from duplicate chunks

---

Sample Questions and RamayanaGPT Advantages over Perplexity.ai

Question	Perplexity.ai Output	RamayanaGPT Output
"Why did Rama kill Vali secretly?"	**Some sources show bias**	**KG clarifies `(Rama, explains_to, Vali)` and `(Vali, accepts, explanation)` with chunk proof from Kishkindha Kanda**
"Why did dasaratha send Rama to the jungle willingly?"	**False positives, tried to fit in the answer**	**Identified `(Kikeyi, aided, dasratha), (Kikeyi, received, two wishes)` and `(dasaratha,crumbled on,Rama's exile)`**

"Who cursed Ravana and why?"	**Mentions a curse but lacks origin details**	**Traces to Vedavati incident, citing the triplet: `(Ravana, misbehaved_with, Vedavati)` and `(Vedavati, cursed, Ravana)`**
"Was Hanuman aware of his powers?"	**Vague or mixed interpretation**	**Identifies `(Hanuman, forgets_powers, due_to_curse)` and `(Jambavan, reminds, Hanuman)` with chunk evidence**

Note: perplexity is evolving daily, these responses were analyzed in March 2025.

Main Technical Challenges Still Remaining

1. **KG Memory Optimization**:
    - All KGs together have ~5k nodes/edges due to repetitions
    - Hard to keep in-memory for efficient traversal
2. **Triplet-Chuck Mapping Collisions**:
    - Same triplet exists in multiple paragraphs
    - Difficult to determine which chunk is most relevant during answer generation
3. **Context Switching**:
    - Hard to preserve core narrative when queries involve multiple characters across multiple kandas
4. **LLM hallucinations**:
    - When the prompt length exceeds 10k tokens, sometimes LLM ignores the prompt line that states 'strictly answer from provided context' and hallucinates sometimes.

Minor Technical Issues and Future Improvements

Issue	Suggested Change
RAG context sometimes irrelevant	Add chunk relevance scoring via cross-attention model
Efficient mini model to assign weights for retrieval from 7 different KGs	Sometimes all the answer is present in only one or two particular kandas
LLM adds hallucinations when huge context and unsure	Use retrieval confidence scores to enforce abstention or fallback answers
No visualization of answers	Add graph visual output showing reasoning path via nodes and edges / backtrack context